# DEVOPS WITH AWS

# MINI PROJECT DOCUMENTATION

Containerizing and Deploying OpsMate Service Using Docker and AWS

**Name:** Vrunda Koli

**Institute:** IT Vedant

**Course:** DevOps with AWS

**Project stack:** Node.js • Express • MySQL • Docker • Docker Compose • Amazon ECR • Amazon ECS • AWS Fargate

This document follows the organization and level of detail of the provided DevOps Fundamentals mini-project document. Runtime evidence and screenshots should be added from the actual environment before final submission.

# 1. Project Overview

This project demonstrates how to package a Node.js web application into a Docker image, run it as a container, connect it to a MySQL database using Docker Compose, and prepare the container image for deployment on AWS using Amazon Elastic Container Registry (ECR), Amazon Elastic Container Service (ECS), and AWS Fargate.

OpsMate is a small web service. Its home page displays the application status and attempts to connect to MySQL. When the database connection succeeds, the application creates a visits table if needed, inserts a visit record, and displays the total number of visits. A /health endpoint returns a JSON health response for the web service.

## Project objectives

• Understand Docker images, containers, ports, networks, and volumes.

• Build a container image for a Node.js/Express application.

• Run the application and MySQL database as separate services with Docker Compose.

• Store application source code in GitHub.

• Push the Docker image to Amazon ECR.

• Run the container using ECS with AWS Fargate and troubleshoot deployment connectivity.

• Document commands, issues, security considerations, and future improvements.

# 2. Technologies Used

| Technology | Purpose in the project |
|---|---|
| Ubuntu on Amazon EC2 | Linux environment used for project setup and Docker commands. |
| Node.js 20 Alpine | Runtime base image used by the Dockerfile. |
| Express.js | Handles HTTP requests and application routes. |
| MySQL 8.4 | Database used to store visit records. |
| Docker Engine | Builds images and runs containers. |
| Docker Compose | Defines and runs the app and MySQL services together. |
| Git and GitHub | Version control and remote source-code repository. |
| Amazon ECR | Stores the Docker image in AWS. |
| Amazon ECS | Schedules and manages container tasks and services. |
| AWS Fargate | Runs ECS tasks without managing container host servers. |

# 3. Architecture

Local container architecture:

```
Browser / HTTP client
        |
        | http://&lt;host&gt;:5000
        v
OpsMate Node.js + Express container
        |
        | Docker network: opsmate-network
        | MySQL port 3306
        v
MySQL 8.4 container
        |
        v
Named volume: mysql-data
```

AWS deployment flow:

```
Source code
```

```
    |
    v
Docker build
    |
    v
Amazon ECR (stores image)
    |
    v
Amazon ECS service / task definition
    |
    v
AWS Fargate task
    |
    v
Application endpoint (networking + security rules required)
```

ECR stores the image; ECS manages the deployment; Fargate supplies the container compute. A working deployment also requires correct task port mappings, subnet/routing configuration, and security group rules.

# 4. Application Components

## 4.1 Project files

| File | Description |
|---|---|
| app/src/server.js | Express server, home page route, MySQL visit tracking, and /health route. |
| app/package.json | Application metadata and Express/mysql2 dependencies. |
| Dockerfile | Instructions for building the application image. |
| docker-compose.yml or compose.yaml | Defines app and MySQL services, network, volume, ports, and health check. Use one canonical file name. |
| README.md | Project summary and setup/use instructions. |
| .dockerignore | Excludes unnecessary files from the Docker build context. |
| scripts/ | Planned automation scripts for installation, image build, running, and cleanup. |
| screenshots/ | Actual screenshots of Docker and AWS configuration. |
| documentation/ | Architecture diagram and final report. |

## 4.2 Application endpoints

| Endpoint | Expected behavior |
|---|---|
| GET / | Shows OpsMate status and attempts MySQL visit-count operations. |
| GET /health | Returns JSON such as {"status":"healthy","service":"opsmate"}. It checks app response, not database readiness. |

# 5. Dockerfile Explanation

```
FROM node:20-alpine
WORKDIR /app
COPY app/package*.json ./
RUN npm install --omit=dev
COPY app/src ./src
EXPOSE 5000
CMD ["npm", "start"]
```

| Instruction | Explanation |
|---|---|
| FROM node:20-alpine | Uses Node.js 20 on a lightweight Alpine Linux base image. |
| WORKDIR /app | Sets the working directory inside the image. |
| COPY app/package*.json ./ | Copies package metadata so dependencies can be installed. |

| Instruction | Explanation |
|---|---|
| RUN npm install --omit=dev | Installs production dependencies. |
| COPY app/src ./src | Copies the application source code. |
| EXPOSE 5000 | Documents the application port; it does not publish the port on its own. |
| CMD ["npm", "start"] | Starts the Node.js application when the container runs. |

# 6. Docker Compose Configuration

The Compose configuration defines two services: app and mysql. The app is built from the Dockerfile and publishes host port 5000 to container port 5000. The MySQL service uses mysql:8.4 and initializes a database named opsmate.

• A bridge network named opsmate-network allows the app to resolve the database by service name mysql.

• A named volume mysql-data stores MySQL files beyond the lifecycle of the database container.

• The MySQL health check uses mysqladmin ping.

• depends_on with service_healthy delays app startup until the MySQL health check succeeds.

• restart: unless-stopped asks Docker to restart services unless they have been explicitly stopped.

**Security note:** The previously shared Compose configuration uses MYSQL_ROOT_PASSWORD=root and the root account from the application. This should be treated as a lab-only configuration. Use a strong secret and a least-privilege database user before any production use.

# 7. Installation and Deployment Steps

## Step 1: Connect to the Ubuntu host

```
ssh -i &lt;key-file.pem&gt; ubuntu@&lt;EC2-public-IP&gt;
cd ~/OpsMate-Docker-Project
```

Use the access method approved for your lab. Do not upload the private key to GitHub.

## Step 2: Verify Docker and Compose

```
docker --version
docker compose version
sudo systemctl status docker --no-pager
docker info
```

Capture the command output for evidence. If Docker cannot contact the daemon, check systemd status and daemon logs.

## Step 3: Validate Compose configuration

```
docker compose config
```

Run this from the directory containing the selected Compose YAML file. If both docker-compose.yml and compose.yaml exist, use one canonical file and avoid conflicting configurations.

## Step 4: Build and start the services

```
docker compose up -d --build
docker compose ps
```

The first command builds the app image if needed and starts the configured services in detached mode. Confirm both services are running and MySQL becomes healthy.

## Step 5: Test the application

```
curl -i http://localhost:5000/
curl -i http://localhost:5000/health
docker compose logs --tail 100
```

The home page should show OpsMate status. Check its database message separately; a successful /health response alone does not prove MySQL connectivity.

### Step 6: Inspect images, containers, and resources

```
docker images --digests
docker ps -a
docker stats --no-stream
docker system df
```

Record the image tag/ID, container status, CPU/memory use, and Docker disk usage in the final report.

# 8. Amazon ECR Image Upload

Amazon ECR is the container image registry used to store the image for cloud deployment. The account ID and region previously shared for this project were 746931976614 and ap-south-1 (Mumbai). Verify that these values still match the account and region in use.

```
aws sts get-caller-identity
aws ecr describe-repositories --repository-names opsmate --region ap-south-1

aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin \
  746931976614.dkr.ecr.ap-south-1.amazonaws.com

docker build -t opsmate:1.0 .
docker tag opsmate:1.0 \
  746931976614.dkr.ecr.ap-south-1.amazonaws.com/opsmate:1.0
docker push 746931976614.dkr.ecr.ap-south-1.amazonaws.com/opsmate:1.0

aws ecr describe-images --repository-name opsmate --region ap-south-1
```

The commands assume that the ECR repository exists and AWS CLI credentials/role are configured. Use an IAM role or least-privilege credentials. Do not include passwords or ECR authentication tokens in documentation.

# 9. Amazon ECS and AWS Fargate Deployment

ECS uses a task definition to describe the container image, ports, environment, CPU/memory, IAM roles, and logging. An ECS service maintains the desired task count. With Fargate, AWS manages the underlying container host infrastructure.

Previously shared resource identifiers were cluster opsmate-cluster, service opsmate-task-service-hpikaa30, and task definition opsmate-task:3. These must be verified before use.

### Commands to inspect deployment

```
aws ecs describe-clusters --clusters opsmate-cluster --region ap-south-1
aws ecs describe-services --cluster opsmate-cluster \
  --services opsmate-task-service-hpikaa30 --region ap-south-1
aws ecs list-tasks --cluster opsmate-cluster \
  --service-name opsmate-task-service-hpikaa30 --region ap-south-1
```

Use the task ARN returned by list-tasks to inspect the task with describe-tasks. Check desiredStatus, lastStatus, stoppedReason, container exit code, image digest, log configuration, and network details.

### Important port configuration

The application listens on port 5000. A previously inspected task definition used container port 80, which does not match the application. Set the ECS container port mapping to 5000, register a new task definition revision, update the service, and wait for deployment stabilization. Also verify the security group, subnet route to an Internet Gateway or load balancer path, and public IP/load balancer configuration. The previous browser test timed out, so public reachability must be retested before calling deployment successful.

# 10. GitHub Version Control

The project was pushed successfully to the GitHub repository currently named Docker-project1. The local folder is named OpsMate-Docker-Project. The initial push failed once because GitHub does not accept account passwords for Git operations; a subsequent push succeeded.

```
git status
git add README.md Dockerfile app compose.yaml .dockerignore config scripts screenshots documentation
git commit -m "Add OpsMate Docker project documentation"
git push origin main
```

Run git add only for files that exist. Review git status before committing. Never commit .env files, AWS credentials, private keys, or real passwords. If the repository is renamed on GitHub, update the origin URL with git remote set-url origin .

## 11. Monitoring and Verification Commands

| Purpose | Command |
|---|---|
| Docker daemon | sudo systemctl is-active docker |
| App logs | docker logs --tail 100 opsmate-app |
| MySQL logs | docker logs --tail 100 opsmate-mysql |
| Compose service status | docker compose ps |
| Container resource usage | docker stats --no-stream |
| Disk and memory | df -h; free -h |
| Docker disk usage | docker system df |
| App HTTP test | curl -i http://localhost:5000/health |
| ECR image list | aws ecr describe-images --repository-name opsmate --region ap-south-1 |
| ECS service | aws ecs describe-services --cluster opsmate-cluster --services opsmate-task-service-hpikaa30 --region ap-south-1 |

## 12. Challenges Faced

| Issue | Troubleshooting / next action |
|---|---|
| GitHub authentication failed | Retry with a GitHub personal access token or use SSH authentication; the later push succeeded. |
| AWS application timed out | Check task state, port mapping, public IP or load balancer, subnet routing, security group, and logs. |
| ECS port mismatch | Application listens on 5000 while the previously inspected task definition mapped 80; update the task definition to 5000. |
| Database connection may fail | Check Compose service health, DB_HOST=mysql, credentials, database name, and MySQL logs. |
| Repository structure still in progress | Create the planned scripts and documentation; add real screenshots rather than empty image placeholders. |
| Hard-coded root password | Replace with secret configuration and a least-privilege database user. |

## 13. Security Considerations

• Do not commit .env files, private keys, AWS credentials, database passwords, or ECR tokens.

• Replace the hard-coded MySQL root password and use a dedicated application database user with limited permissions.

• Use AWS IAM least privilege and an EC2 instance role where appropriate.

• Use AWS Secrets Manager or Systems Manager Parameter Store for deployed secrets.

• Do not expose MySQL port 3306 to the public internet.

• Restrict inbound application traffic to the necessary sources and port; use HTTPS via an Application Load Balancer and TLS certificate for production.

• Keep base images and Node dependencies updated, scan images, and configure log retention.

• Consider running the Node.js process as a non-root user in the container.

## 14. Resource Optimization and Improvements

• Commit package-lock.json and use npm ci --omit=dev for repeatable production dependency installation.

• Add and verify a .dockerignore file to keep the build context small.

• Use image tags tied to a version or commit SHA instead of relying only on latest.

• Set ECS task CPU and memory using measured resource usage.

• Add CloudWatch log retention, metrics, and alarms for failed deployments and task restarts.

• Add a distinct readiness check that validates database availability; keep liveness checks lightweight.

• Use a MySQL connection pool and avoid creating a table on every home-page request; manage schema through migrations.

• Create and verify automation scripts for building, running, and cleaning up containers.

• Add an architecture diagram, screenshots, and a README with exact setup and troubleshooting instructions.

## 15. Folder Structure

```
OpsMate-Docker-Project/
■■■ README.md
■■■ Dockerfile
■■■ compose.yaml                # or docker-compose.yml; keep one canonical file
■■■ .dockerignore
■■■ docker-deployment-report.txt
■■■ app/
■   ■■■ package.json
■   ■■■ src/
■       ■■■ server.js
■■■ config/
■   ■■■ environment.example
■■■ scripts/
■   ■■■ install-docker.sh
■   ■■■ build-image.sh
■   ■■■ run-container.sh
■   ■■■ cleanup.sh
■■■ screenshots/
■   ■■■ docker-installation.png
■   ■■■ docker-daemon.png
■   ■■■ docker-image.png
■   ■■■ docker-container.png
■   ■■■ application-running.png
■   ■■■ docker-compose.png
■   ■■■ ecr-repository.png
■   ■■■ ecr-image.png
■   ■■■ ecs-cluster.png
■   ■■■ ecs-task.png
■   ■■■ fargate-application.png
■■■ documentation/
    ■■■ Docker-Architecture.png
    ■■■ Project_Report.pdf
```

This is the recommended target structure. The screenshot and PDF files listed are expected deliverables; only include them once real evidence and final documents have been created. Do not create empty files with .png or .pdf extensions.

## 16. Learning Outcomes

• Understood how Docker images are built and containers are run.

• Learned how a Node.js application is configured to listen on a container port.

• Learned to define an app and database using Docker Compose.

• Practiced container networking, port mapping, health checks, and persistent volumes.

• Learned the roles of ECR, ECS, and Fargate in a container deployment workflow.

• Practiced Git/GitHub version control and troubleshooting authentication.

• Identified the importance of application logs, resource metrics, IAM, secrets, and network security.

# 17. Project Outcome

The OpsMate application code, Dockerfile, and Docker Compose configuration have been created, and the source repository was successfully pushed to GitHub. AWS ECR/ECS/Fargate resources were previously configured, but the application browser test returned a timeout and a port mismatch was identified in the historical task definition. The project should be marked fully deployed only after correcting the task definition and networking, checking task/service health, and successfully testing the endpoint.

For final submission, add dated screenshots and current command outputs for Docker installation, daemon status, image/container details, resource usage, logs, Compose status, ECR image details, ECS service status, Fargate task status, and the working application endpoint.

# 18. Evidence Checklist

| Evidence | What to capture | Ready? |
|---|---|---|
| Docker installation | docker --version and docker compose version | Pending |
| Docker daemon | systemctl status docker | Pending |
| Image details | docker images --digests / inspect | Pending |
| Containers | docker ps -a | Pending |
| Resource usage | docker stats --no-stream | Pending |
| Application logs | docker logs output | Pending |
| Compose status | docker compose ps | Pending |
| ECR | ECR repository and image details | Pending verification |
| ECS service/task | Current service and task status | Pending verification |
| Fargate application | Successful browser or curl response | Retest required |
| GitHub repository | Repository page and final commit | Initial push confirmed |

Reference basis: This documentation is modeled on the section structure and presentation style of the supplied “DevOps Fundamentals MINI Project” document, with content adapted to the OpsMate Docker project and clearly distinguishing known configuration from evidence that still needs to be collected.